# LESSON 07

# 没好 while 继续

## while 循环：条件成立就一直做

学完你能写一个猜对才结束的小游戏。

# 猜数字，要猜几次?

小明

**猜 1 次**

小红

**猜 5 次**

小刚

**猜 12 次**

写程序的时候，你不知道别人要猜几次，range 里填几都不对。
这时要换一种说法：只要还没猜对，就继续。

# while：条件成立就继续

count.py

```
1 i = 1
2 while i <= 3:
3     print(i)
4     i = i + 1
```

再问：4 <= 3 不成立，循环结束。

while 是“当……的时候”。条件成立就运行缩进的行，运行完回到 while 再问一次。这一行末尾也有冒号。

| | |
|---|---|
| i = 1 | **1** |
| i = 2 | **2** |
| i = 3 | **3** |
| i = 4 | 结束 |

# 猜一猜：把 if 换成 while

写 if

```
i = 1
if i <= 3:
    print(i)
    i = i + 1
```

**1**

写 while

```
i = 1
while i <= 3:
    print(i)
    i = i + 1
```

**1 2 3**

if 只问一次，成立就做一遍。while 做完还会回来再问。

# 写 while 要想好三件事

## 1

### 开始值

`i = 1`

变量从哪里开始

## 2

### 继续条件

`i <= 3`

什么时候还要继续

## 3

### 更新

`i = i + 1`

每一遍怎么往前走

这三件事 for 里的 range 帮你管着，用 while 就要自己写。

# 小测：最后一行输出几？

输出几？

```
1 i = 1
2 while i <= 3:
3       i = i + 1
4 print(i)
```

1 3

2 4

3 1

4 报错

答对了！i 变成 4 时条件才不成立，循环这才停下，所以是 4。

# 同一件事，两种写法

**for：从 1 数到 n**

```
n = int(input())
for i in range(1, n + 1):
    print(i)
```

**while：从 1 数到 n**

```
n = int(input())
i = 1
while i <= n:
    print(i)
    i = i + 1
```

知道要重复几次，用 for 更短。

要等一件事发生才停，比如猜对、读到 0，用 while。

# 小测：一直读数，读到 0 才停，用哪个？

1 while

2 for

3 if

4 print

答对了！不知道要读几次，只知道什么时候停，正适合 while。

# 动手改一改

三件事都要改：开始值、条件和更新。

## 用 while 倒计时

读入一个 1 到 100 的整数 n，从 n 倒数到 1，每个数占一行，最后输出发射！

输入 2，输出

```
2
1
发射！
```

输入 1，输出

```
1
发射！
```

教室电脑：打开 starter/03_countdown.py

# 忘了更新，就停不下来

只阅读，不要运行

```
1 i = 1
2 while i <= 3:
3       print(i)
```

屏幕上，一直输出 1

1

1

1

……

i 一直是 1，条件一直成立，程序就停不下来了。这叫**死循环**。

万一遇到了：在 IDLE 里按 Ctrl + C。在线 IDE 就刷新网页，代码要重新写。

运行之前问问自己：每一遍哪个值变了？这样变下去，条件最后会不成立吗？

# 小测：填哪一句，循环会停不下来?

空白处填一句

```
1 i = 5
2 while i > 0:
3         ……
```

1 `i = i - 1`

2 `i = i - 2`

3 `i = i + 1`

4 `i = 0`

答对了！i 越来越大，离 0 越来越远，i > 0 一直成立。有更新还不够，要往结束的方向走。

# 例题：猜对才结束

guess.py

```
1 secret = 7
2 guess = int(input())
3 while guess != secret:
4     print("不对，再猜")
5     guess = int(input())
6 print("猜对了")
```

input 写了两次：循环前读第一次，循环里读下一次。

| | |
|---|---|
| 猜 3 | 不对 |
| 猜 9 | 不对 |
| 猜 7 | 猜对了 |

# 猜一猜：第一次就输入 7

guess.py

```
1 secret = 7
2 guess = int(input())
3 while guess != secret:
4     print("不对，再猜")
5     guess = int(input())
6 print("猜对了")
```

屏幕上，只有一行

**猜对了**

一次也没有。while 先判断，再运行。条件一开始就不成立，循环一遍也不运行，直接走到后面。

# 动手改一改

while 里面也能放 if，缩进要跟上。

## 提示大了还是小了

答案还是 7。每行读入一个 1 到 100 的整数，猜大了输出大了，猜小了输出小了，猜中才结束。

| 依次输入 | 输出 |
| --- | --- |
| 50 | 大了 |
| 3 | 小了 |
| 7 | 猜对了 |

教室电脑：打开 starter/05_guess_hint.py

# # 这节课常见的三种错误

划线的写法只阅读，不要照抄

| | | |
|---|---|---|
| ~~`while i <= 3`~~ | `SyntaxError` | 末尾要加冒号 |
| ~~`while i = 3:`~~ | `SyntaxError` | 比较要写 `==` |
| 循环前忘了写 `i = 1` | `NameError` | 先给开始值 |

要小心：忘了更新变量不报错，只是停不下来。

# 找错：怎么一个数也没输出？

bug.py（只阅读）

```
1 i = 10
2 while i < 0:
3     print(i)
4     i = i - 1
5 print("结束")
```

想从 10 倒数到 1，程序却只输出结束，也没有报错。
点你认为有问题的那一行。

点代码里的一行

# 自己挑战

# 反着念数字

读入 n（1 到 1000000 的整数），从个位开始，一位一位地输出，每位占一行。
输入 100，就输出 0、0、1 三行。

输入一行

**123**

输出三行

**3**

**2**

**1**

先自己写，卡住了再翻下一页。

# 卡住了？一次看一条提示

**提示 1**　数有几位事先不知道，所以用 while。怎么取出个位？第 3 课学过。

**提示 2**　123 // 10 得到 12，个位就去掉了。把结果放回 n。

**提示 3**　n 变成多少时该停？试试输入 7 和 100。

# # 造一个答对才结束的游戏

程序一直问，答对了才放你走。题目你自己定。

### 先做到

用 `while` 一直读回答，答对了才结束。

### 选你喜欢的

猜数字、猜谜语、背口诀、密码锁，什么都行。

### 还想更厉害

答错时给提示，再连着出好几道题。

没有标准答案。先说清它要做什么，再试两组输入，一组试边界。

# # 和同桌换着玩

## 1

### 换着闯关

玩同桌的游戏，看几次能答对

## 2

### 说一个喜欢的

告诉同桌，哪一句提示最有意思

## 3

### 提一个想法

答错的时候，程序怎样说才更好？

听完再改一版，让游戏更好玩。

# 说得清吗?

| | | |
|---|---|---|
| Q1 | while 和 if 有什么不同? | if 只问一次，while 反复问。 |
| Q2 | 写 while 要想好哪三件事? | 开始值、继续条件、更新。 |
| Q3 | 什么时候用 while 更合适? | 不知道要重复几次的时候。 |

# 速查卡

| | |
|---|---|
| `while 条件:` | 条件成立就运行缩进的行，再回来问 |
| `i = 1` | 循环前先给开始值 |
| `i = i + 1` | 循环里要更新，朝结束的方向走 |
| `guess != secret` | 不相等就继续，猜对才停 |
| `Ctrl + C` | IDLE 里停下死循环 |

# 这些后面再学

`total = total + i`

用循环把一串数加起来

`i += 1`

更新变量的简写

`break`

在循环中间提前跳出来

今天程序学会了等条件。下节课用循环把数加起来。

# 谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 python-for-youth

下一课：累加和计数